



AMRITA

VISHWA VIDYAPEETHAM

SUBMITTED BY:

NAME:	M. KARTHIK REDDY
ROLL NO:	CSE25037
YEAR / SEM / SECTION	2 / 3 / CSE-A
CREDITS	5

SUBMITTED TO:

NAME:	Dr. RAJKUMAR BATCU
DEPARTMENT	SCHOOL OF COMPUTING
DESIGNATION	ASST. PROFFESOR (SI grade)

S NO:	TITLE	PAGE	REMARKS
1	WEEK 1: 1a: Recursive countdown from n 1b: Recursive calculation of compound growth factor 1c: Recursive search for emo id in list 1d: Recursive factorial calculation 1e: Recursive fibonacci series		
2	WEEK 2: 2a: Linear Search Program 2b: Binary Search Program		
3	WEEK 3: 3a: Bubble Sort 3b: Insertion Sort 3c: Selection Sort 3d: Merge Sort 3e: Quick Sort		
4	WEEK 4: 4a: Singly Linked List 4b: Doubly Linked List 4c: Circular Linked List		
5	WEEK 5: 5a: Stack Using Array 5b: Stack Using Linked List		
6	WEEK 6: 6a: Queue Using Array 6b: Queue Using Linked List 6c: Circular Queue		

LAB 1

1. AIM:

A rocket launch system displays a countdown. Use no loops. Write a Python program to display the countdown from n to 1, followed by "Launch".

PROGRAM:

```
def countdown(n):
    print(n)
    if n==0:
        return 0
    else:
        return countdown(n-1)

n=int(input("enter number"))
print(countdown(n))
```

OUTPUT:

```
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: /Users/karthikreddy/Documents/SEM 3/DSA/lab/week 1/1.py =====
enter number9
9
8
7
6
5
4
3
2
1
0
0
>>>
```

EXPLANATION:

- Defines a recursive countdown() function that prints numbers in decreasing order.
- Prints the current value of n before making the recursive call.
- When n becomes 0, the recursion stops.
- Otherwise, the function calls itself with n-1.
- Takes the starting number as input and displays the countdown.

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	Countdown does not stop	Check the condition when n reaches 0
2	Program stops for a very large number	Use a loop instead of recursion
3	Negative numbers are printed unexpectedly	Allow only non negative starting values

2. AIM:

Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate p^n (power), where p = principal growth factor and n = number of years.

PROGRAM:

```
def power(p,n):
    if n<=0:
        return 1
    else:
        return p*power(p,n-1)
```

```
p=int(input("Enter prinipal"))
n=int(input("Enter number"))
Print(power(p,n))
```

OUTPUT:

[illegible]

EXPLANATION:

- Defines a recursive `power()` function to calculate (p^n) .
- Returns 1 when $n \leq 0$, which is the base case.
- Otherwise, multiplies p by `power(p, n-1)` repeatedly.
- Takes the base (p) and exponent (n) as input.
- Calls the function and prints the calculated power.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Incorrect result for negative exponent	Add separate handling for negative exponents
2	Program becomes slow for large exponent	Use an efficient power calculation method
3	Wrong result for exponent zero	Make sure the base case returns 1

3. AIM:

An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

PROGRAM:

```
def search(A,k,n=0):
    if n>=len(A):
        return -1
    if A[n]==k:
        return n
    return search(A,k,n+1)

A=[1,2,3,4,5,6,7,8]
print(A)
k=int(input("enter id to search"))
```

Print(search(A,k))

OUTPUT:

```
>>> |===== RESTART: /Users/karthikreddy/Documents/SEM 3/DSA/lab/week 1/3.py =====|
      | [1, 2, 3, 4, 5, 6, 7, 8]|
      | enter id to search5|
      | 4|
      |>>>|
```

EXPLANATION:

- Defines a recursive search() function to find the target element in the list.
- Checks each element starting from index 0 using recursion.
- If the target is found, its index is returned.
- If the end of the list is reached, -1 is returned.
- Takes the target value as input and displays the search result.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Target element is not found	Check whether the target exists in the list
2	Recursion stops before checking all elements	Verify the index increment in the recursive call
3	Program stops for a large list	Use an iterative search method

4. AIM:

In a logistics company, the number of possible ways to arrange n different parcels is calculated using the factorial formula ($n!$). Develop a recursive program to calculate the factorial of a given number.

PROGRAM:

```
def factorial(n):
    if n<=0:
        return 1
    else:
        return n*factorial(n-1)

n=int(input("Enter number"))
print(factorial(n))
```

OUTPUT:

```
>>> ===== RESTART: /Users/karthikreddy/Documents/SEM 3/DSA/lab/week 1/4.py =====
Enter number7
5040
>>>
```

EXPLANATION:

- Defines a recursive factorial() function to calculate the factorial of a number.
- Returns 1 when $n \leq 0$, which also handles $0!$.
- Otherwise, multiplies n by factorial($n-1$) until reaching the base case.
- Takes a number as input from the user.
- Calls the function and prints the factorial result.

ERROR TABLE:

S.No	Error Obtained	Rectification
------	----------------	---------------

1	Incorrect factorial for negative input	Restrict the input to non negative numbers
2	Program stops for a large number	Use an iterative approach
3	Wrong factorial result	Check the recursive multiplication and base condition

5. AIM:

Write a recursive Python program to print the first n terms of the Fibonacci series.

PROGRAM:

```
def fibonacci(n):
    if n<=0:
        return 0
    elif n==1:
        return 1
    else:
        return fibonacci(n-1)+fibonacci(n-2)
```

```
n= int(input("Enter terms"))
for i in range(n):
    print(fibonacci(i))
```

OUTPUT:

```
>>> ===== RESTART: /Users/karthikreddy/Documents/SEM 3/DSA/lab/week 1/5.py =====
Enter terms8
0
1
1
2
3
5
8
13
>>>|
```

EXPLANATION:

- Defines a recursive fibonacci() function to calculate Fibonacci numbers.
- Returns 0 for $n \leq 0$ and 1 for $n == 1$.
- For other values, it recursively adds the previous two Fibonacci numbers.
- Takes the number of terms from the user.
- Uses a loop to generate and print each Fibonacci term.
- The sequence starts as 0, 1, 1, 2, 3, 5,

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Program runs very slowly	Use an iterative or optimized approach
2	Incorrect sequence is displayed	Check the base conditions for 0 and 1
3	Program stops for large input	Avoid excessive recursive calls

LAB 2

1. AIM:

Write a program to perform linear search on a given input.

PROGRAM:

```
def linearSearch(a,key):
    for i in range(n):
        if a[i]==key:
            return i
    return -1

n=int(input("Enter number of elements"))
a=[]
for i in range(n):
    x=int(input("enter elements"))
    a.append(x)
key=int(input("Enter target element"))
Print(linearSearch(a,key))
```

OUTPUT:

```
>>>|===== RESTART: /Users/karthikreddy/Documents/SEM 3/DSA/lab/week 2/1.py =====|
Enter number of elements5
enter elements2
enter elements3
enter elements4
enter elements5
enter elements9
Enter target element4
2
>>>|
```

EXPLANATION:

- Takes the number of elements and stores them in a list.
- Takes the target element to be searched.
- linearSearch() checks each element one by one.
- If the target is found, its index is returned.
- If not found, -1 is returned.
- The result is displayed using print().

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Element is not found	Check whether the required element is present in the list
2	Wrong index is displayed	Check the list indexing starting from 0

2. AIM:

Write a Python program to perform binary search on the given input. Verify by taking sorted array & unsorted array as input.

PROGRAM:

```
def binary_search(a, low, high, key):
    while low <= high:
        mid = (low + high) // 2

        if a[mid] == key:
            return mid
        elif a[mid] > key:
            high = mid - 1
        else:
            low = mid + 1

    return -1

n = int(input("Enter number of elements: "))
if n <= 0:
    print("Number of elements must be positive.")
a = []
for i in range(n):
    x = int(input("Enter element: "))
    a.append(x)
a.sort()
print("Sorted list:",a)
key = int(input("Enter target element: "))
result = binary_search(a, 0, len(a) - 1, key)
if result != -1:
    print("Element found at index",result)
else:
    print("Element not found.")
```

OUTPUT:

```
>>>
===== RESTART: /Users/karthikreddy/Documents/SEM 3/DSA/lab/week 2/2.py =====
Enter number of elements: 5
Enter element: 3
Enter element: 6
Enter element: 7
Enter element: 1
Enter element: 9
Sorted list: [1, 3, 6, 7, 9]
Enter target element: 7
Element found at index 3
>>>
```

EXPLANATION:

- Takes the number of elements and stores them in a list.
- Sorts the list using `a.sort()` because binary search requires sorted data.
- Calculates the middle element and compares it with the target.
- If the target is smaller or larger, searches only the appropriate half.
- Returns the index if found; otherwise returns -1.
- Displays whether the element is found or not.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Wrong element is returned	Make sure the list is sorted before searching
2	Search gives incorrect index	Check the calculation of the middle index
3	Element is not found	Verify the low and high boundaries

LAB 3

1. AIM:

Write a program to sort the elements using a bubble sort technique. Take an input elements from user

PROGRAM:

```
def bubble_sort(arr):
    n=len(arr)
    for i in range(n-1):
        for j in range(n-i-1):
            if arr[j]>arr[j+1]:
                arr[j],arr[j+1]=arr[j+1],arr[j]
    return arr
num=int(input("Enter number of elements: "))
arr=[]
for i in range(num):
    x=int(input("Enter element: "))
    arr.append(x)
print(arr)
print("Sorted array is: ",bubble_sort(arr))
```

OUTPUT:

```
===== RESTART: /Users/karthikreddy/Documents/SEM 3/DSA/lab/week 3/1.py =====
Enter number of elements: 6
Enter element: 6
Enter element: 2
Enter element: 5
Enter element: 9
Enter element: 0
Enter element: 7
[6, 2, 5, 9, 0, 7]
Sorted array is: [0, 2, 5, 6, 7, 9]
```

EXPLANATION:

- Defines the bubble_sort function and finds the number of elements in the list.
- Compares adjacent elements using two nested loops.
- Swaps the elements if they are in the wrong order.
- Repeats the process until all elements are arranged in ascending order.
- Takes the elements as input from the user and stores them in a list.
- Calls bubble_sort and displays the sorted array.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	IndexError while comparing elements	Check the loop range and array size

2	Elements are not sorted correctly	Check the swapping condition
3	Incorrect number of elements processed	Make sure the entered count matches the elements

2. AIM:

Write a program to sort the elements using a insertion sort technique. Take an input elements from user

PROGRAM:

```
def insertion_sort(arr):
    n=len(arr)
    for i in range(1,n):
        key=arr[i]
        j=i-1
        while j>=0 and arr[j]>key:
            arr[j+1]=arr[j]
            j=j-1
        arr[j+1]=key
    return arr
num=int(input("Enter number of elements: "))
arr=[]
for i in range(num):
    x=int(input("Enter element: "))
    arr.append(x)
print(arr)
print("Sorted array is: ", insertion_sort(arr))
```

OUTPUT:

```
===== RESTART: /Users/karthikreddy/Documents/SEM 3/DSA/lab/week 3/2.py =====
Enter number of elements: 5
Enter element: 2
Enter element: 18
Enter element: 7
Enter element: 45
Enter element: 10
[2, 18, 7, 45, 10]
Sorted array is: [2, 7, 10, 18, 45]
```

EXPLANATION:

- Defines the insertion_sort function and starts from the second element of the list.
- Stores the current element in key and compares it with the previous elements.
- Moves larger elements one position to the right.
- Places key in its correct position in the sorted portion of the list.
- Repeats the process until all elements are sorted.
- Displays the original and sorted arrays.

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	Elements are not sorted correctly	Check the position where key is inserted
2	IndexError while shifting elements	Check the condition $j \geq 0$
3	Some elements are overwritten	Place key after completing the shifting process

3. AIM:

Write a program to sort the elements using a selection sort technique. Take an input elements from user

PROGRAM:

```
def selection_sort(arr):
    n=len(arr)
    for i in range(n-1):
        min_index=i
        for j in range(i+1,n):
            if arr[j]<arr[min_index]:
                min_index=j
        arr[i] , arr[min_index] = arr[min_index] , arr[i]
    return arr
num=int(input("Enter number of elements: "))
arr=[]
for i in range(num):
    x=int(input("Enter element: "))
    arr.append(x)
print("Sorted array is: ", selection_sort(arr))
```

OUTPUT:

```
===== RESTART: /Users/karthikreddy/Documents/SEM 3/DSA/lab/week 3/3.py =====
Enter number of elements: 4
Enter element: 9
Enter element: 4
Enter element: 1
Enter element: 0
Sorted array is: [0, 1, 4, 9]
```

EXPLANATION:

- Defines the selection_sort function and assumes the first unsorted element as the minimum.
- Compares it with the remaining elements to find the smallest element.
- Stores the position of the smallest element in min_index.
- Swaps the smallest element with the element at the current position.

- Repeats the process until the entire list is sorted.
- Takes input elements and displays the sorted array.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Incorrect sorting of elements	Perform the swap after finding the minimum element
2	Elements are swapped multiple times	Move the swap statement outside the inner loop
3	Minimum element is selected incorrectly	Check the min_index update condition

4. AIM:

Write a program to sort the elements using a merge sort technique. Take an input elements from user

PROGRAM:

```
def merge_sort(a):
    if len(a)>1:
        mid=len(a)//2
        left=a[:mid]
        right=a[mid:]
        merge_sort(left)
        merge_sort(right)
        i=j=k=0
        while i<len(left) and j<len(right):
            if left[i]<right[j]:
                a[k]=left[i]
                i+=1
            else:
                a[k]=right[j]
                j+=1
            k+=1
        while i<len(left):
            a[k]=left[i]
            i+=1
            k+=1
        while j<len(right):
            a[k]=right[j]
            j+=1
            k+=1
num=int(input("Enter number of elements: "))
a=[]
for i in range(num):
    x=int(input("Enter element: "))
    a.append(x)
print("Original array:",a)
merge_sort(a)
```

```
print("Sorted array:",a)
```

OUTPUT:

```
===== RESTART: /Users/karthikreddy/Documents/SEM 3/DSA/lab/week 3/4.py =====
Enter number of elements: 5
Enter element: 3
Enter element: 5
Enter element: 6
Enter element: 2
Enter element: 0
Original array: [3, 5, 6, 2, 0]
Sorted array: [0, 2, 3, 5, 6]
```

EXPLANATION:

- Defines the merge_sort function and divides the array into two halves.
- Recursively divides each half until single elements remain.
- Compares elements from the left and right halves.
- Merges the elements back into the array in ascending order.
- Continues merging until the complete array is sorted.
- Takes input elements and displays the original and sorted arrays.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Elements are not merged correctly	Check the comparison between left and right arrays
2	Some elements are missing after merging	Check the remaining element loops
3	Incorrect sorted output	Verify the values of i, j and k during merging

5. AIM:

Write a program to sort the elements using a quick sort technique. Take an input elements from user

PROGRAM:

```
def quicksort(a,low,high):
    if low<high:
        i=low
        j=high
        pivot=low
        while i<j:
            while i<=high and a[i]<=a[pivot]:
                i+=1
            while a[j]>a[pivot]:
                j-=1
            if i<j:
                a[i],a[j]=a[j],a[i]
```

```
a[pivot],a[j]=a[j],a[pivot]
quicksort(a,low,j-1)
quicksort(a,j+1,high)
```

```
num=int(input("Enter number of elements: "))
a=[]
for i in range(num):
    x=int(input("Enter element: "))
    a.append(x)
print(a)
n=len(a)
quicksort(a,0,n-1)
print("Sorted array is:",a)
```

OUTPUT:

```
===== RESTART: /Users/karthikreddy/Documents/SEM 3/DSA/lab/week 3/5.py =====
Enter number of elements: 5
Enter element: 4
Enter element: 7
Enter element: 2
Enter element: 9
Enter element: 8
[4, 7, 2, 9, 8]
Sorted array is: [2, 4, 7, 8, 9]
```

EXPLANATION:

- Defines a recursive Quick Sort function and selects the first element as the pivot.
- Uses two pointers, i and j, to compare and arrange elements around the pivot.
- Swaps elements when they are on the wrong side of the pivot.
- Places the pivot in its correct position after partitioning.
- Recursively sorts the elements before and after the pivot.
- Takes the elements as input and performs the sorting operation.

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	UnboundLocalError for i or j	Keep the partitioning code inside the if block
2	RecursionError for large input	Check the recursive calls and pivot position
3	Sorted array is displayed as None	Print the list after calling quicksort

LAB 4

1. AIM:

Write a python programme to implement a single linked list which could perform the below operations ,by taking an input from the user.

- 1)create a linked list
- 2)Insert at beginning
- 3)Insert at end
- 4)insert at index
- 5)Delete by value
- 6)Delete First Node
- 7)Delete Last Node
- 8)Count number of nodes
- 9)Display/Traverse
- 10)Exit

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class SLL:
    def __init__(self):
        self.head = None

    def create(self):
        n = int(input("Enter number of nodes: "))
        self.head = None
        for i in range(n):
            data = int(input("Enter data: "))
            new_node = Node(data)
            if self.head is None:
                self.head = new_node
            else:
                temp = self.head
                while temp.next is not None:
                    temp = temp.next
                temp.next = new_node
        print("Linked List created successfully.")

    def insert_beginning(self):
        data = int(input("Enter data: "))
        new_node = Node(data)
```

```
new_node.next = self.head
self.head = new_node
print(f"Inserted {data} at the beginning.")
```

```
def insert_end(self):
    data = int(input("Enter data: "))
    new_node = Node(data)
    if self.head is None:
        self.head = new_node
        print(f"Inserted {data} at the end.")
        return
    temp = self.head
    while temp.next is not None:
        temp = temp.next
    temp.next = new_node
    print(f"Inserted {data} at the end.")
```

```
def insert_at_index(self):
    data = int(input("Enter data: "))
    index = int(input("Enter index: "))
    if index < 0:
        print("Invalid index.")
        return
    if index == 0:
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node
        print(f"Inserted {data} at index {index}.")
        return
    temp = self.head
    for i in range(index - 1):
        if temp is None:
            print("Index out of range.")
            return
        temp = temp.next
    if temp is None:
        print("Index out of range.")
        return
    new_node = Node(data)
    new_node.next = temp.next
    temp.next = new_node
    print(f"Inserted {data} at index {index}.")
```

```
def delete_by_value(self):
    value = int(input("Enter value to delete: "))
    if self.head is None:
        print("List is empty.")
        return
    if self.head.data == value:
        self.head = self.head.next
        print(f"Deleted {value}.")
        return
```

```

temp = self.head
while temp.next is not None:
    if temp.next.data == value:
        temp.next = temp.next.next
        print(f"Deleted {value}.")
        return
    temp = temp.next
print("Value not found.")

```

```

def delete_beginning(self):
    if self.head is None:
        print("List is empty.")
        return
    deleted_data = self.head.data
    self.head = self.head.next
    print(f"Deleted {deleted_data} from the beginning.")

```

```

def delete_end(self):
    if self.head is None:
        print("List is empty.")
        return
    if self.head.next is None:
        deleted_data = self.head.data
        self.head = None
        print(f"Deleted {deleted_data} from the end.")
        return
    temp = self.head
    while temp.next.next is not None:
        temp = temp.next
    deleted_data = temp.next.data
    temp.next = None
    print(f"Deleted {deleted_data} from the end.")

```

```

def count_nodes(self):
    count = 0
    temp = self.head
    while temp is not None:
        count += 1
        temp = temp.next
    print("Number of nodes:", count)

```

```

def display(self):
    if self.head is None:
        print("List is empty.")
        return
    temp = self.head
    while temp is not None:
        print(temp.data, end=" -> ")
        temp = temp.next
    print("None")

```

```
sll = SLL()
```

```
while True:
    print("\n----- SINGLY LINKED LIST -----")
    print("1. Create Linked List")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Index")
    print("5. Delete by Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Number of Nodes")
    print("9. Display")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        sll.create()
    elif choice == 2:
        sll.insert_beginning()
    elif choice == 3:
        sll.insert_end()
    elif choice == 4:
        sll.insert_at_index()
    elif choice == 5:
        sll.delete_by_value()
    elif choice == 6:
        sll.delete_beginning()
    elif choice == 7:
        sll.delete_end()
    elif choice == 8:
        sll.count_nodes()
    elif choice == 9:
        sll.display()
    elif choice == 10:
        print("Exiting from the program...")
        break
    else:
        print("Invalid choice.")
```

OUTPUT:

===== RESTART: /Users/karthikreddy/Desktop/SEM 3/DSA/lab/week 4/1.py =====

----- SINGLY LINKED LIST -----

1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit

Enter your choice: 1

Enter number of nodes: 4

Enter data: 2

Enter data: 6

Enter data: 7

Enter data: 5

Linked List created successfully.

----- SINGLY LINKED LIST -----

1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit

Enter your choice: 10

Exiting from the program...

EXPLANATION:

- Defines Node and SLL classes to create and manage a singly linked list.
- The create function creates nodes and connects them using the next pointer.
- Provides operations to insert nodes at the beginning, end, and a specific index.
- Provides operations to delete a node by value, from the beginning, and from the end.
- The count and display functions show the number of nodes and the contents of the list.
- A menu driven loop allows the user to select and perform different linked list operations.

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	IndentationError in class or function definitions	Maintain proper indentation inside classes and functions
2	AttributeError while accessing next	Check that the node is created correctly before accessing next
3	Index out of range during insertion	Enter an index within the valid linked list range

2. AIM:

Write a python program to implement a Double linked list which could perform the below operations ,by taking an input from the user.

1)Insert at beginning

2)Insert at end

3)insert at index

4)Delete by value

5)Delete First Node

6)Delete Last Node

7)Display the Forward Nodes

8)Display the Backward Nodes

9)Exit

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.prev = None
        self.data = data
        self.next = None
```

```
class DoublyLinkedList:
    def __init__(self):
```

```
self.head = None
```

```
def insert_begin(self, data):  
    new = Node(data)  
    if self.head is None:  
        self.head = new  
    else:  
        new.next = self.head  
        self.head.prev = new  
        self.head = new  
    print("Insertion Completed")
```

```
def insert_end(self, data):  
    new = Node(data)  
    if self.head is None:  
        self.head = new  
    else:  
        temp = self.head  
        while temp.next:  
            temp = temp.next  
        temp.next = new  
        new.prev = temp  
    print("Insertion Completed")
```

```
def insert_index(self, index, data):  
    if index < 0:  
        print("Invalid Index")  
    elif index == 0:  
        self.insert_begin(data)  
    else:  
        temp = self.head  
        for i in range(index - 1):  
            if temp is None:  
                print("Invalid Index")  
                return  
            temp = temp.next  
        if temp is None:  
            print("Invalid Index")  
            return  
        new = Node(data)  
        new.next = temp.next  
        new.prev = temp  
        if temp.next:  
            temp.next.prev = new  
        temp.next = new  
    print("Insertion Completed")
```

```
def deleteBeg(self):  
    if self.head is None:  
        print("Can't perform delete operation")  
    elif self.head.next is None:  
        self.head = None  
        print("Value Deleted")  
    else:  
        self.head.next.prev = None  
        self.head = self.head.next  
        print("Value Deleted")
```

```
def deleteEnd(self):  
    if self.head is None:
```

```

        print("Can't perform delete operation")
    elif self.head.next is None:
        self.head = None
        print("Value Deleted")
    else:
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.prev.next = None
        print("Value Deleted")

def deleteIndex(self, index):
    if self.head is None:
        print("Can't perform delete operation")
    elif index < 0:
        print("Invalid Index")
    elif index == 0:
        self.deleteBeg()
    else:
        temp = self.head
        for i in range(index):
            if temp is None:
                print("Invalid Index")
                return
            temp = temp.next
        if temp is None:
            print("Invalid Index")
        elif temp.next:
            temp.next.prev = temp.prev
            temp.prev.next = temp.next
            print("Value Deleted")
        else:
            temp.prev.next = None
            print("Value Deleted")

def displayFwd(self):
    temp = self.head
    while temp:
        print(temp.data, end=" <-> ")
        temp = temp.next
    print("None")

def displayBwd(self):
    if self.head is None:
        print("None")
        return
    temp = self.head
    while temp.next:
        temp = temp.next
    while temp:
        print(temp.data, end=" <-> ")
        temp = temp.prev
    print("None")

DII = DoublyLinkedList()

while True:
    print("\n1.Insert Beginning")
    print("2.Insert End")
    print("3.Insert at Index")

```

```
print("4.Delete at Beginning")
print("5.Delete at End")
print("6.Delete at Specific Index")
print("7.Display Forward")
print("8.Display Backward")
print("9.Exit")
```

```
ch = int(input("Choice: "))
```

```
if ch == 1:
    x = int(input("Value: "))
   Dll.insert_begin(x)
    Dll.displayFwd()
elif ch == 2:
    x = int(input("Value: "))
    Dll.insert_end(x)
    Dll.displayFwd()
elif ch == 3:
    idx = int(input("Index: "))
    x = int(input("Value: "))
    Dll.insert_index(idx, x)
    Dll.displayFwd()
elif ch == 4:
    Dll.deleteBeg()
    Dll.displayFwd()
elif ch == 5:
    Dll.deleteEnd()
    Dll.displayFwd()
elif ch == 6:
    idx = int(input("Index: "))
    Dll.deleteIndex(idx)
    Dll.displayFwd()
elif ch == 7:
    Dll.displayFwd()
elif ch == 8:
    Dll.displayBwd()
elif ch == 9:
    print("Loop Terminated")
    break
else:
    print("Invalid Choice")
```

OUTPUT:

===== RESTART: /Users/karthikreddy/Desktop/SEM 3/DSA/lab/week 4/2.py =====

```
1.Insert Beginning
2.Insert End
3.Insert at Index
4.Delete at Beginning
5.Delete at End
6.Delete at Specific Index
7.Display Forward
8.Display Backward
9.Exit
Choice: 1
Value: 4
Insertion Completed
4 <-> None
```

```
1.Insert Beginning
2.Insert End
3.Insert at Index
4.Delete at Beginning
5.Delete at End
6.Delete at Specific Index
7.Display Forward
8.Display Backward
9.Exit
Choice: 6
Index: 1
Invalid Index
4 <-> None
```

```
1.Insert Beginning
2.Insert End
3.Insert at Index
4.Delete at Beginning
5.Delete at End
6.Delete at Specific Index
7.Display Forward
8.Display Backward
9.Exit
Choice: 9
Loop Terminated
```

EXPLANATION:

- Defines a Node class with data, previous link and next link for a doubly linked list.
- Provides functions to insert nodes at the beginning, end and a specific index.
- Provides functions to delete nodes from the beginning, end and a specific index.
- Maintains both prev and next links while inserting and deleting nodes.
- The display functions show the list in forward and backward directions.
- A menu allows the user to perform different doubly linked list operations.

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	Invalid Index during insertion	Enter an index within the valid list range
2	Links are not updated correctly after deletion	Update both prev and next links properly
3	Backward display gives incorrect order	Traverse to the last node before moving backward

3. AIM:

Write a python program to implement a Circular linked list which could perform the below operations ,by taking an input from the user.

- 1)create a linked list**
- 2)Insert at beginning**
- 3)Insert at end**
- 4)insert at index**
- 5)Delete by value**
- 6)Delete First Node**
- 7)Delete Last Node**
- 8)Count number of nodes**
- 9)Display**
 - a)Head to Tail**
 - b)Tail to Head**
- 10)Exit**

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularLinkedList:
    def __init__(self):
        self.head = None

    def create(self):
        n = int(input("Enter number of nodes: "))
        self.head = None
        for i in range(n):
            data = int(input("Enter data: "))
            new_node = Node(data)
```

```

        if self.head is None:
            self.head = new_node
            new_node.next = self.head
        else:
            temp = self.head
            while temp.next != self.head:
                temp = temp.next
            temp.next = new_node
            new_node.next = self.head
    print("Circular Linked List created successfully.")

def insert_beginning(self):
    data = int(input("Enter data: "))
    new_node = Node(data)
    if self.head is None:
        self.head = new_node
        new_node.next = self.head
    else:
        temp = self.head
        while temp.next != self.head:
            temp = temp.next
        new_node.next = self.head
        temp.next = new_node
        self.head = new_node
    print(f"Inserted {data} at the beginning.")

def insert_end(self):
    data = int(input("Enter data: "))
    new_node = Node(data)
    if self.head is None:
        self.head = new_node
        new_node.next = self.head
    else:
        temp = self.head
        while temp.next != self.head:
            temp = temp.next
        temp.next = new_node
        new_node.next = self.head
    print(f"Inserted {data} at the end.")

def insert_at_index(self):
    index = int(input("Enter index: "))
    data = int(input("Enter data: "))
    if index < 0:
        print("Invalid index.")
        return
    if index == 0:
        self.insert_beginning()
        return
    if self.head is None:
        print("Index out of range.")
        return
    temp = self.head
    for i in range(index - 1):
        temp = temp.next
        if temp == self.head:
            print("Index out of range.")
            return
    new_node = Node(data)
    new_node.next = temp.next

```

```

temp.next = new_node
print(f"Inserted {data} at index {index}.")

def delete_by_value(self):
    value = int(input("Enter value to delete: "))
    if self.head is None:
        print("Circular Linked List is empty.")
        return
    if self.head.data == value:
        if self.head.next == self.head:
            self.head = None
        else:
            temp = self.head
            while temp.next != self.head:
                temp = temp.next
            temp.next = self.head.next
            self.head = self.head.next
        print(f"Deleted {value}.")
        return
    temp = self.head
    while temp.next != self.head:
        if temp.next.data == value:
            temp.next = temp.next.next
            print(f"Deleted {value}.")
            return
        temp = temp.next
    print(f"{value} not found in the Linked List.")

def delete_first(self):
    if self.head is None:
        print("Circular Linked List is empty.")
        return
    deleted = self.head.data
    if self.head.next == self.head:
        self.head = None
    else:
        temp = self.head
        while temp.next != self.head:
            temp = temp.next
        temp.next = self.head.next
        self.head = self.head.next
    print(f"Deleted first node: {deleted}")

def delete_last(self):
    if self.head is None:
        print("Circular Linked List is empty.")
        return
    if self.head.next == self.head:
        deleted = self.head.data
        self.head = None
        print(f"Deleted last node: {deleted}")
        return
    temp = self.head
    while temp.next.next != self.head:
        temp = temp.next
    deleted = temp.next.data
    temp.next = self.head
    print(f"Deleted last node: {deleted}")

def count_nodes(self):

```

```

if self.head is None:
    print("Number of nodes: 0")
    return
count = 0
temp = self.head
while True:
    count += 1
    temp = temp.next
    if temp == self.head:
        break
print("Number of nodes:", count)

```

```

def display(self):
    if self.head is None:
        print("Circular Linked List is empty.")
        return
    temp = self.head
    print("Circular Linked List:", end=" ")
    while True:
        print(temp.data, end=" -> ")
        temp = temp.next
        if temp == self.head:
            break
    print("HEAD")

```

```

def display_head_tail(self):
    if self.head is None:
        print("Circular Linked List is empty.")
        return
    temp = self.head
    while temp.next != self.head:
        temp = temp.next
    print("Head:", self.head.data)
    print("Tail:", temp.data)

```

```

def display_tail_to_head(self):
    if self.head is None:
        print("Circular Linked List is empty.")
        return
    tail = self.head
    while tail.next != self.head:
        tail = tail.next
    print("Data from Tail to Head:", end=" ")
    current = tail
    while True:
        print(current.data, end=" ")
        if current == self.head:
            break
        temp = self.head
        while temp.next != current:
            temp = temp.next
        current = temp
    print()

```

```

c11 = CircularLinkedList()

```

```

while True:
    print("\n----- CIRCULAR LINKED LIST -----")
    print("1. Create Linked List")
    print("2. Insert at Beginning")

```

```
print("3. Insert at End")
print("4. Insert at Specific Index")
print("5. Delete by Value")
print("6. Delete First Node")
print("7. Delete Last Node")
print("8. Count Number of Nodes")
print("9. Display")
print("10. Display Head and Tail")
print("11. Print Data from Tail to Head")
print("12. Exit")
```

```
choice = int(input("Enter your choice: "))
```

```
if choice == 1:
    cll.create()
elif choice == 2:
    cll.insert_beginning()
elif choice == 3:
    cll.insert_end()
elif choice == 4:
    cll.insert_at_index()
elif choice == 5:
    cll.delete_by_value()
elif choice == 6:
    cll.delete_first()
elif choice == 7:
    cll.delete_last()
elif choice == 8:
    cll.count_nodes()
elif choice == 9:
    cll.display()
elif choice == 10:
    cll.display_head_tail()
elif choice == 11:
    cll.display_tail_to_head()
elif choice == 12:
    print("Exiting program....")
    break
else:
    print("Invalid choice. Please try again.")
```

OUTPUT:

```
===== RESTART: /Users/karthikreddy/Desktop/SEM 3/DSA/lab/week 4/3.py =====

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Display Head and Tail
11. Print Data from Tail to Head
12. Exit
Enter your choice: 1
Enter number of nodes: 3
Enter data: 2
Enter data: 7
Enter data: 9
Circular Linked List created successfully.

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Display Head and Tail
11. Print Data from Tail to Head
12. Exit
Enter your choice: 9
Circular Linked List: 2 -> 7 -> 9 -> HEAD
```

EXPLANATION:

- Defines Node and CircularLinkedList classes where the last node points back to the head.
- Provides functions to create and insert nodes at the beginning, end and a specific index.
- Provides functions to delete nodes by value, from the beginning and from the end.
- Count and display functions traverse the list until they reach the head again.
- Additional functions display the head and tail and traverse the list from tail to head.
- A menu allows the user to perform different circular linked list operations.

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	Infinite loop while traversing the list	Stop traversal when the current node reaches the head
2	Incorrect head after deletion	Update the head and last node link correctly
3	Index out of range during insertion	Enter an index within the valid list range

LAB 5

1. AIM:

Write a python program to implement a stack using an array, also implement the stack operations such as:

a)push

b)pop

c)peek also display the stack elements

PROGRAM:

```
class StackEx:
    def __init__(self, size):
        self.size = size
        self.stack = [None] * size
        self.top = -1

    def push(self, item):
        if self.top == self.size - 1:
            print("Stack Overflow")
        else:
            self.top += 1
            self.stack[self.top] = item
            print(item, "pushed into the stack")

    def pop(self):
        if self.top == -1:
            print("Stack Underflow")
        else:
            item = self.stack[self.top]
            self.stack[self.top] = None
            self.top -= 1
            print(item, "popped from the stack")

    def peek(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("Top element:", self.stack[self.top])

    def display(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("The elements of the stack are:")
            for i in range(self.top, -1, -1):
                print(self.stack[i])
```

```
size = int(input("Enter the size of the stack: "))
s = StackEx(size)

while True:
    print("\n----- STACK MENU -----")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to push: "))
        s.push(item)
    elif choice == 2:
        s.pop()
    elif choice == 3:
        s.peek()
    elif choice == 4:
        s.display()
    elif choice == 5:
        print("Program terminated.")
        break
    else:
        print("Invalid choice")
```

OUTPUT:

===== RESTART: /Users/karthikreddy/Desktop/SEM 3/DSA/lab/week 5/1.py =====

Enter the size of the stack: 3

----- STACK MENU -----

1. Push
2. Pop
3. Peek
4. Display
5. Exit

Enter your choice: 1

Enter the element to push: 7

7 pushed into the stack

----- STACK MENU -----

1. Push
2. Pop
3. Peek
4. Display
5. Exit

Enter your choice: 1

Enter the element to push: 18

18 pushed into the stack

----- STACK MENU -----

1. Push
2. Pop
3. Peek
4. Display
5. Exit

Enter your choice: 3

Top element: 18

----- STACK MENU -----

1. Push
2. Pop
3. Peek
4. Display
5. Exit

Enter your choice: 5

Program terminated.

EXPLANATION:

- Defines a StackEx class that creates a stack with a fixed size and initializes top as -1.
- The push function adds an element to the top and checks for stack overflow.
- The pop function removes the top element and checks for stack underflow.
- The peek function displays the current top element without removing it.
- The display function prints all stack elements from top to bottom.
- A menu allows the user to perform push, pop, peek, display and exit operations.

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	Stack Overflow occurs when adding elements	Increase the stack size or remove existing elements
2	Stack Underflow occurs while popping	Check whether the stack contains elements before popping
3	IndentationError in class or menu section	Maintain proper indentation throughout the program

2. AIM:

Write a python program to implement a stack using linked list ,also implement the stack operations such as:

a)push

b)pop

c)peek also display the stack elements

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Stack:
    def __init__(self):
        self.top = None

    def push(self, data):
        new = Node(data)
        new.next = self.top
        self.top = new
        print(data, "pushed into stack")

    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            temp = self.top
            print(temp.data, "popped from stack")
            self.top = self.top.next

    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)

    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            temp = self.top
            print("Stack elements:")
            while temp is not None:
                print(temp.data)
                temp = temp.next
```

```
s = Stack()

while True:
    print("\n--- STACK USING LINKED LIST ---")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter the element: "))
        s.push(data)
    elif choice == 2:
        s.pop()
    elif choice == 3:
        s.peek()
    elif choice == 4:
        s.display()
    elif choice == 5:
        print("Program terminated")
        break
    else:
        print("Invalid choice")
```

OUTPUT:

===== RESTART: /Users/karthikreddy/Desktop/SEM 3/DSA/lab/week 5/1.py =====

Enter the size of the stack: 3

----- STACK MENU -----

1. Push
2. Pop
3. Peek
4. Display
5. Exit

Enter your choice: 1

Enter the element to push: 7

7 pushed into the stack

----- STACK MENU -----

1. Push
2. Pop
3. Peek
4. Display
5. Exit

Enter your choice: 1

Enter the element to push: 18

18 pushed into the stack

----- STACK MENU -----

1. Push
2. Pop
3. Peek
4. Display
5. Exit

Enter your choice: 4

The elements of the stack are:

18

7

EXPLANATION:

- Defines a Node class to store data and the link to the next node.
- The Stack class uses top to keep track of the top node.
- The push function creates a new node and adds it to the top of the stack.
- The pop function removes the top node and handles stack underflow.
- The peek and display functions show the top element and all stack elements.
- A menu allows the user to perform different stack operations.

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	Stack Underflow while popping	Check whether the stack is empty before popping
2	AttributeError while accessing node data	Check that top points to a valid node
3	IndentationError in class or menu section	Maintain proper indentation throughout the program

LAB 6

1. AIM:

Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu-driven approach. Implement using array data structures.

PROGRAM:

```
class Queue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, x):
        if self.rear == self.size - 1:
            print("Queue Overflow")
        else:
            if self.front == -1:
                self.front = 0
            self.rear += 1
            self.queue[self.rear] = x
            print(f"{x} inserted into the queue")

    def dequeue(self):
        if self.front == -1 or self.front > self.rear:
            print("Queue Underflow")
        else:
            x = self.queue[self.front]
            self.queue[self.front] = None
            self.front += 1
            print(f"{x} deleted from the queue")

            if self.front > self.rear:
                self.front = -1
                self.rear = -1

    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Front element:", self.queue[self.front])

    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            for i in range(self.front, self.rear + 1):
                print(self.queue[i])

size = int(input("Enter queue size: "))
q = Queue(size)

while True:
```

```

print("1. Enqueue")
print("2. Dequeue")
print("3. Peek")
print("4. Display")
print("5. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:
    x = int(input("Enter element: "))
    q.enqueue(x)

elif choice == 2:
    q.dequeue()

elif choice == 3:
    q.peek()

elif choice == 4:
    q.display()

elif choice == 5:
    print("Program terminated")
    break

else:
    print("Invalid choice")

```

OUTPUT:

```

===== RESTART: /Users/karthikreddy/Desktop/SEM 3/DSA/lab/week 6/1.py =====
Enter queue size: 3
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 3
3 inserted into the queue
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 3
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated

```

EXPLANATION:

- Defines a Queue class using a fixed size list and initializes front and rear as -1.
- The enqueue function adds elements at the rear and checks for queue overflow.
- The dequeue function removes elements from the front and checks for queue underflow.
- The peek function displays the front element without removing it.
- The display function prints all elements from front to rear.
- A menu allows the user to perform enqueue, dequeue, peek, display and exit operations.

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	Queue Overflow when inserting an element	Increase the queue size or remove existing elements
2	Queue Underflow while deleting an element	Check whether the queue contains elements before deletion
3	Elements are not displayed correctly	Check the front and rear values before displaying

2. AIM:

Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu-driven approach. Implement using linked list data structures.

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class QueueEx:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self, x):
        new = Node(x)

        if self.rear is None:
            self.front = self.rear = new
        else:
            self.rear.next = new
            self.rear = new

        print(x, "inserted into the queue")

    def dequeue(self):
        if self.front is None:
            print("Queue Underflow")
```

```
else:
    x = self.front.data
    self.front = self.front.next

    if self.front is None:
        self.rear = None

    print(x, "deleted from the queue")
```

```
def peek(self):
    if self.front is None:
        print("Queue is empty")
    else:
        print("Front element:", self.front.data)
```

```
def display(self):
    if self.front is None:
        print("Queue is empty")
    else:
        print("The elements of the queue are:")

        temp = self.front
        while temp is not None:
            print(temp.data)
            temp = temp.next
```

```
q = QueueEx()
```

```
while True:
    print("\n----- QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()

    elif choice == 4:
        q.display()

    elif choice == 5:
        print("Program terminated.")
        break

    else:
        print("Invalid choice")
```

OUTPUT:

```
===== RESTART: /Users/karthikreddy/Desktop/SEM 3/DSA/lab/week 6/2.py =====

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 7
7 inserted into the queue

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 18
18 inserted into the queue

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
The elements of the queue are:
7
18
```

EXPLANATION:

Defines a Node class to store the queue element and the link to the next node.

QueueEx uses front and rear pointers to manage the queue.

The enqueue function adds a new node at the rear of the queue.

The dequeue function removes a node from the front and handles an empty queue.

The peek and display functions show the front element and all queue elements.

A menu allows the user to perform enqueue, dequeue, peek, display and exit operations.

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	Queue Underflow during dequeue	Check whether front is None before deleting
2	Rear pointer becomes incorrect	Set rear to None when the last node is removed
3	Elements are not displayed correctly	Check the next link while traversing the queue

3. AIM:

Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu-driven approach.

PROGRAM:

```

class CircularQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, item):
        if (self.rear + 1) % self.size == self.front:
            print("Queue Overflow")
        elif self.front == -1:
            self.front = 0
            self.rear = 0
            self.queue[self.rear] = item
            print(item, "inserted into the queue")
        else:
            self.rear = (self.rear + 1) % self.size
            self.queue[self.rear] = item
            print(item, "inserted into the queue")

    def dequeue(self):
        if self.front == -1:
            print("Queue Underflow")
        elif self.front == self.rear:
            item = self.queue[self.front]
            self.queue[self.front] = None
            self.front = -1
            self.rear = -1
            print(item, "deleted from the queue")
        else:
            item = self.queue[self.front]
            self.queue[self.front] = None
            self.front = (self.front + 1) % self.size
            print(item, "deleted from the queue")

    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            i = self.front
            while True:
                print(self.queue[i])
                if i == self.rear:
                    break
                i = (i + 1) % self.size

size = int(input("Enter the size of the queue: "))
q = CircularQueue(size)

while True:
    print("\n----- CIRCULAR QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Display")
    print("4. Exit")

    choice = int(input("Enter your choice: "))

```

```

if choice == 1:
    item = int(input("Enter the element to enqueue: "))
    q.enqueue(item)
elif choice == 2:
    q.dequeue()
elif choice == 3:
    q.display()
elif choice == 4:
    print("Program terminated.")
    break
else:
    print("Invalid choice")

```

OUTPUT:

```

===== RESTART: /Users/karthikreddy/Desktop/SEM 3/DSA/lab/week 6/3.py =====
Enter the size of the queue: 4

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter the element to enqueue: 3
3 inserted into the queue

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter the element to enqueue: 6
6 inserted into the queue

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
3 deleted from the queue

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 4
Program terminated.

```

EXPLANATION:

- Defines a CircularQueue class using a fixed size list with front and rear pointers.
- The enqueue function inserts elements and uses modulo to move rear circularly.
- The dequeue function removes elements from the front and updates the front position.
- The queue checks for overflow when the next rear position reaches the front.
- The display function starts from front and moves circularly until it reaches rear.
- A menu allows the user to perform enqueue, dequeue, display and exit operations.

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	Queue Overflow occurs unexpectedly	Check the circular full condition using front and rear
2	Queue Underflow while deleting	Check whether front is set to -1 before deletion
3	Incorrect elements are displayed	Check the circular movement using modulo operation

